Bab 4 Konsep dan Implementasi Multi-Tenancy

Tujuan Pembelajaran

Setelah mempelajari bab ini, Anda mampu:

1. **(C2 – Memahami)** menjelaskan konsep *multi-tenancy* serta perbedaan strategi *shared database–shared schema*, *shared database–separate schema*, dan *separate database*.
2. **(C3 – Menerapkan)** menerapkan *tenant context*, *middleware*, *global scope*, dan *policy* untuk membatasi akses data berdasarkan tenant.
3. **(C4 – Menganalisis)** menganalisis risiko kebocoran data lintas tenant pada aplikasi dengan arsitektur *shared schema*.
4. **(C5 – Mengevaluasi)** mengevaluasi efektivitas isolasi tenant melalui skenario pengujian yang dapat diulang.
5. **(C6 – Mencipta)** merancang pendekatan isolasi multi-tenant yang sesuai dengan kebutuhan skala, kustomisasi, dan keamanan aplikasi.

Orientasi Bab

Pada aplikasi *Kantin Multi-Tenant*, satu sistem melayani banyak kantin sekaligus. Setiap tenant memiliki menu, pesanan, pengguna, dan laporan sendiri. Tantangan utamanya bukan sekadar menyimpan data dalam tabel yang sama, melainkan memastikan data Kantin A tidak dapat dibaca, diubah, atau dihapus oleh pengguna Kantin B.

Masalah ini penting karena satu filter `tenant_id` yang terlewat dapat membuka data milik tenant lain. Oleh sebab itu, isolasi tenant tidak boleh hanya bergantung pada ketelitian pengembang saat menulis kueri. Aplikasi perlu dirancang agar aman secara bawaan (*secure by default*) dan menolak akses ketika identitas tenant tidak dapat dipastikan (*fail-closed*).

Bab ini membahas pilihan arsitektur multi-tenant, penerapan konteks tenant, penyaringan kueri dengan *global scope*, otorisasi berbasis kepemilikan, serta pengujian kebocoran lintas tenant.

Peta bab. Seluruh contoh kode pada bab ini diambil dari proyek pendamping *Kantin Multi-Tenant* (tag `pertemuan-04`), sehingga nama kelas, namespace, dan perilaku yang Anda baca di sini sama dengan yang Anda jalankan di praktikum. Seluruh angka, diagram, dan studi kasus bersifat **ilustratif**.

Prasyarat. Bab ini menggunakan konsep *service container*, *middleware*, dan *global scope* yang telah dibahas pada bab sebelumnya. Jika Anda belum memahami ketiganya, pelajari kembali Bab 2 dan Bab 3 sebelum menerapkan kode pada bab ini.

4.1 Mengapa Isolasi Tenant Diperlukan

Multi-tenancy adalah pola arsitektur ketika satu instans aplikasi melayani banyak penyewa atau *tenant*. Dalam bab ini, tenant berupa satu tenant kantin, yaitu satu unit usaha yang berjualan di dalam sebuah kantin.

Berbeda dengan *single-tenancy*, aplikasi multi-tenant berbagi sebagian sumber daya: kode program, server aplikasi, atau basis data. Berbagi sumber daya dapat menekan biaya dan

mempermudah pemeliharaan. Namun, berbagi juga menimbulkan risiko: kesalahan pada satu bagian aplikasi dapat menyebabkan data tenant lain terlihat atau dapat diubah.

Pada aplikasi *Kantin Multi-Tenant*, tabel `menus` menyimpan menu dari seluruh tenant. Karena itu, setiap baris data milik tenant harus memiliki penanda kepemilikan, yaitu kolom `tenant_id`.

id	tenant_id	nama	harga
1	1	Nasi Goreng Tenant A	15.000
2	1	Es Teh Tenant A	4.000
3	2	Mie Ayam Tenant B	14.000
4	2	Jus Jeruk Tenant B	8.000

Tabel 4.1. Contoh 4.1 — data menu dari dua tenant pada satu tabel bersama (ilustratif).

Apabila pengguna dari tenant 1 menjalankan kueri tanpa filter `tenant_id`, aplikasi akan mengembalikan keempat baris sekaligus. Dalam kondisi tersebut, dua menu milik tenant 2 telah terekspos.



Gambar 4.1. Perbandingan pembacaan data tanpa dan dengan penyaringan tenant.

Sumber: diadaptasi penulis, ilustratif.

Isolasi tenant harus diterapkan pada beberapa lapisan sekaligus. Penyaringan kueri membatasi data yang dapat **ditemukan**. *Policy* membatasi **tindakan** terhadap data yang telah ditemukan. Pengujian memastikan kedua mekanisme itu benar-benar bekerja ketika kode berubah.

4.1.1 Ukuran Ilustratif Dampak Kebocoran

Untuk memberi gambaran tentang besarnya dampak, kita dapat memakai model sederhana berikut.

$$L = R \cdot \frac{T - 1}{T} \qquad \text{(Persamaan 4.1)}$$

dengan L adalah perkiraan jumlah baris milik tenant lain yang ikut terekspos, R adalah jumlah seluruh baris pada tabel bersama, dan T adalah jumlah tenant. Model ini mengasumsikan data tersebar merata antartenant.

Misalkan sebuah tabel `menus` memuat 1.000 baris dari 20 tenant. Dengan Persamaan 4.1 diperoleh $L = 1.000 \times 19/20 = 950$ baris. Artinya, satu kueri tanpa filter berpotensi mengekspos 950 baris milik tenant lain.

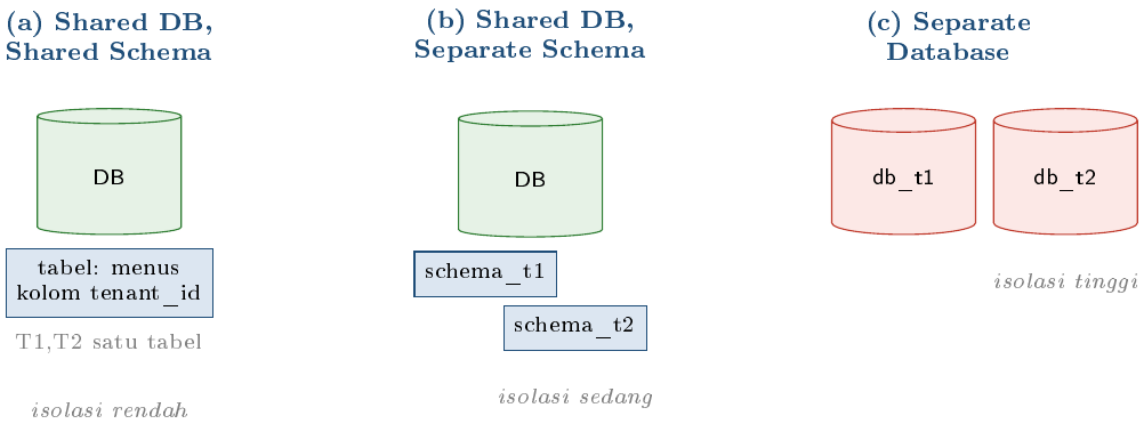
Peringatan. Persamaan 4.1 adalah model pembelajaran dengan asumsi distribusi data yang merata. Distribusi nyata hampir selalu timpang, dan dampak sesungguhnya bergantung pada jenis data yang terekspos, bukan hanya jumlah barisnya.

4.2 Memilih Arsitektur Multi-Tenant

Sebelum menerapkan isolasi pada kode, Anda perlu memilih cara menyimpan data tenant. Tiga strategi yang paling umum adalah *shared database–shared schema*, *shared database–separate schema*, dan *separate database*.

4.2.1 Tiga Strategi Penyimpanan Data

- **Shared database–shared schema.** Seluruh tenant menggunakan basis data dan tabel yang sama. Pemisahan dilakukan melalui kolom seperti `tenant_id` pada setiap baris data milik tenant.
- **Shared database–separate schema.** Tenant masih berbagi satu server atau satu basis data, tetapi masing-masing memiliki skema atau kumpulan tabel tersendiri.
- **Separate database.** Setiap tenant menggunakan basis data terpisah. Isolasi pada tingkat penyimpanan lebih kuat, tetapi pengelolaan migrasi, cadangan, pemantauan, dan pemulihan menjadi lebih kompleks.



Gambar 4.2. Tiga strategi arsitektur multi-tenant: shared schema, separate schema, dan separate database.

Sumber: diadaptasi penulis, ilustratif.

Aspek	Shared schema	Separate schema	Separate database
Lokasi data tenant	Tabel yang sama	Skema berbeda	Basis data berbeda
Bentuk isolasi utama	Logis, melalui kolom dan kontrol aplikasi	Pemisahan struktur skema	Pemisahan basis data
Biaya operasional awal	Cenderung rendah	Cenderung menengah	Cenderung tinggi
Pengelolaan migrasi	Satu skema utama	Dilakukan pada banyak skema	Dilakukan pada banyak basis data
Kustomisasi struktur	Terbatas	Lebih fleksibel	Paling fleksibel
Risiko salah	Tinggi bila filter aplikasi	Tetap ada pada konfigurasi	Tetap ada pada kredensial

konfigurasi	lemah	akses	dan koneksi

Tabel 4.2. Perbandingan umum strategi arsitektur multi-tenant.

Peringatan. Tabel 4.2 bukan aturan mutlak. *Separate database* tidak otomatis menjamin tidak ada kebocoran data: kredensial yang terlalu luas, kesalahan pemilihan koneksi, atau proses ekspor yang tidak terkontrol tetap dapat menyebabkan insiden.

4.2.2 Menentukan Strategi yang Sesuai

Pemilihan strategi perlu mempertimbangkan setidaknya empat faktor: jumlah tenant, kebutuhan isolasi, kebutuhan kustomisasi, dan kemampuan operasional tim.

Kebutuhan dominan	Strategi yang dapat dipertimbangkan	Alasan
Banyak tenant kecil dengan struktur data seragam	Shared database–shared schema	Efisien dalam penggunaan sumber daya dan migrasi
Tenant memerlukan variasi struktur tabel	Separate schema atau separate database	Lebih memungkinkan perubahan struktur per tenant
Kebutuhan audit atau pemisahan data sangat tinggi	Separate database	Pemisahan penyimpanan lebih kuat, tetapi menuntut operasi yang matang
Tim kecil dengan kemampuan operasional terbatas	Shared schema dengan kontrol aplikasi ketat	Lebih sederhana dikelola, tetapi wajib disertai pengujian isolasi yang disiplin

Tabel 4.3. Pertimbangan pemilihan strategi arsitektur multi-tenant.

Pada aplikasi *Kantin Multi-Tenant*, strategi *shared database–shared schema* dipilih karena banyak tenant kantin menggunakan fitur dan struktur data yang sama. Konsekuensinya jelas: aplikasi harus memastikan setiap kueri dan tindakan selalu berada dalam tenant yang benar. Sisa bab ini membahas cara memenuhi kewajiban tersebut.

4.2.3 Model Sederhana Beban Operasional

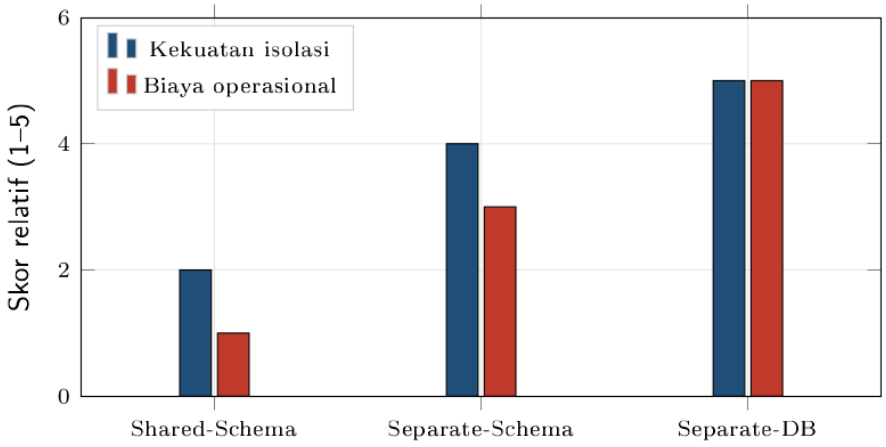
Untuk membantu memahami perbedaan beban operasional antarstrategi, kita dapat memakai model ilustratif berikut.

$$O_{\text{shared}} \approx b, \qquad O_{\text{separate}} = T \cdot b \tag{Persamaan 4.2}$$

dengan O adalah perkiraan jumlah unit pekerjaan operasional, b adalah beban dasar satu unit operasi (misalnya satu pelaksanaan migrasi atau satu pencadangan), dan T adalah jumlah unit penyimpanan yang harus dikelola.

Pada *shared schema*, jumlah unit penyimpanan mendekati satu karena seluruh tenant memakai satu skema utama. Pada *separate database*, jumlah unit penyimpanan dapat sama dengan jumlah tenant.

Contoh 4.2. Misalkan terdapat 50 tenant dan beban dasar satu migrasi adalah 1 unit. Pada *shared schema* diperoleh $O \approx 1$ unit; pada *separate database* diperoleh $O = 50 \times 1 = 50$ unit.



Gambar 4.3. Perbandingan relatif kekuatan isolasi dan biaya operasional pada tiga strategi (skor relatif 1–5).

Sumber: diadaptasi penulis, data ilustratif.

Gambar 4.3 merangkum pertukaran (*trade-off*) yang menjadi inti subbab ini: kekuatan isolasi dan biaya operasional cenderung bergerak searah. Skor pada gambar tersebut bersifat relatif dan ilustratif, bukan hasil pengukuran.

Peringatan. Persamaan 4.2 hanya model pembelajaran. Model ini tidak menghitung biaya permintaan, kapasitas penyimpanan, otomatisasi *deployment*, atau beban pemantauan. Jangan menggunakannya sebagai satu-satunya dasar anggaran atau keputusan arsitektur produksi.

4.3 Fondasi Isolasi pada Shared Schema

Pada arsitektur *shared schema*, seluruh tenant berada pada tabel yang sama. Karena itu, isolasi tidak dapat diserahkan sepenuhnya pada struktur basis data. Aplikasi harus mengetahui tenant aktif dan menggunakan informasi tersebut secara konsisten.

Dua prinsip menjadi dasar pembahasan berikut:

1. setiap sumber daya milik tenant harus dapat dihubungkan secara jelas dengan `tenant_id`;
2. aplikasi harus menolak operasi apabila tenant aktif tidak dapat ditentukan.

Prinsip kedua disebut *fail-closed*. Dalam sistem multi-tenant, kondisi “tenant belum diketahui” tidak boleh diartikan sebagai “tampilkan semua data”.



Gambar 4.4. Empat lapis pertahanan isolasi tenant pada aplikasi Kantin Multi-Tenant.

Sumber: diadaptasi penulis, ilustratif.

Gambar 4.4 menunjukkan bahwa isolasi tenant tidak bertumpu pada satu mekanisme. Keempat lapisan itu dibahas berurutan pada Subbab 4.3 sampai 4.5, lalu dibuktikan melalui pengujian pada Subbab 4.6.

4.3.1 Tenant Context

Tenant context adalah objek yang menyimpan identitas tenant aktif selama satu siklus kerja aplikasi. Pada permintaan HTTP, siklus tersebut dimulai ketika permintaan masuk dan berakhir ketika respons dikirimkan. Objek ini memungkinkan model, *policy*, dan layanan aplikasi memakai satu sumber identitas tenant yang sama, sehingga pengembang tidak perlu meneruskan `tenant_id` secara manual ke setiap metode.

Pada proyek pendamping, kelas ini berada di berkas `app/Support/Tenancy/TenantContext.php`.

Kode 4.1 — app/Support/Tenancy/TenantContext.php

```php
<?php

namespace App\Support\Tenancy;

use App\Models\Tenant;
use RuntimeException;

final class TenantContext
{
    private ?Tenant $tenant = null;

    public function set(Tenant $tenant): void
    {
        $this->tenant = $tenant;
    }

    public function clear(): void
    {
        $this->tenant = null;
    }

    public function has(): bool
    {
        return $this->tenant !== null;
    }

    /** Gagal cepat bila context belum diisi. */
    public function tenant(): Tenant
    {
        return $this->tenant
            ?? throw new RuntimeException('TenantContext belum diisi.');
    }

    public function id(): int
    {
        return $this->tenant()->id;
    }

    public function idOrNull(): ?int
    {
        return $this->tenant?->id;
    }
}
```

Perhatikan pembagian tugas keempat metode pembaca. Metode `has()` dipakai ketika kode perlu mengetahui apakah konteks sudah tersedia. Metode `tenant()` dan `id()` bersifat *fail-fast*:

keduanya melemparkan pengecualian ketika konteks kosong, sehingga cocok untuk operasi yang tidak boleh berjalan tanpa tenant aktif. Metode `idOrNull()` mengembalikan `null` tanpa pengecualian dan dipakai pada pembandingan di *policy*.

Metode `clear()` menghapus konteks setelah satu unit kerja selesai. Hal ini penting pada proses jangka panjang seperti *queue worker*, agar tenant dari pekerjaan sebelumnya tidak terbawa ke pekerjaan berikutnya.

4.3.2 Daur Hidup Konteks: scoped, Bukan singleton

`TenantContext` harus hidup tepat satu siklus permintaan atau satu *job*. Karena itu, proyek pendamping mendaftarkannya sebagai *scoped binding* pada berkas `app/Providers/AppServiceProvider.php`, bukan sebagai *singleton*.

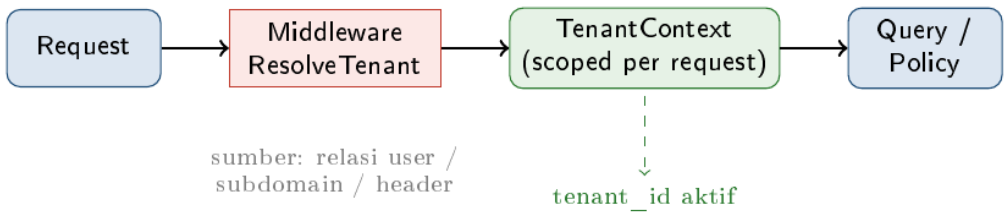
Kode 4.2 — app/Providers/AppServiceProvider.php (potongan register())

```php
public function register(): void
{
    // Scoped: satu instance per request/job; direset di lifecycle
    // berikutnya sehingga tidak bocor antar-request.
    $this->app->scoped(
        TenantContext::class,
        fn (): TenantContext => new TenantContext,
    );
}
```

Catatan. Perbedaan `singleton()` dan `scoped()` baru terasa pada *runtime* yang menahan aplikasi di memori antar-permintaan, seperti *queue worker* jangka panjang, Laravel Octane, atau server Reverb. Pada *runtime* seperti itu, sebuah `singleton()` dapat membawa tenant dari permintaan sebelumnya. Dengan `scoped()`, Laravel melepaskan instans tersebut pada awal siklus berikutnya.

4.3.3 Resolusi Tenant melalui Middleware

Tenant resolution adalah proses menentukan tenant yang sedang aktif. Proses ini ditempatkan pada *middleware* agar berjalan sebelum controller, *policy*, dan kueri aplikasi.



Gambar 4.5. Alur resolusi tenant dari permintaan hingga kueri dan policy.

Sumber: diadaptasi penulis, ilustratif.

Pada proyek pendamping, tenant tidak ditentukan dari atribut pengguna, melainkan dari parameter `slug` pada URL — lalu diverifikasi terhadap keanggotaan pengguna. *Middleware* ini berada di berkas `app/Http/Middleware/SetTenantContext.php`.

Kode 4.3 — app/Http/Middleware/SetTenantContext.php

```php
<?php

namespace App\Http\Middleware;
```

```php
use App\Models\Tenant;
use App\Models\UserTenantRole;
use App\Support\Tenancy\TenantContext;
use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

final class SetTenantContext
{
    public function handle(Request $request, Closure $next): Response
    {
        $tenant = $request->route('tenant');
        if (! $tenant instanceof Tenant) {
            $tenant = Tenant::query()
                ->where('slug', (string) $tenant)->first();
        }

        if (! $tenant instanceof Tenant) {
            abort(404);
        }
        if (! $tenant->isActive()) {
            abort(403);
        }

        $user = $request->user();
        if ($user === null) {
            abort(403);
        }

        $isMember = UserTenantRole::query()
            ->where('user_id', $user->id)
            ->where('tenant_id', $tenant->id)
            ->exists();
        if (! $isMember) {
            abort(403);
        }

        $context = app(TenantContext::class);
        $context->set($tenant);

        try {
            return $next($request);
        } finally {
            $context->clear();
        }
    }
}
```

Alur kerjanya adalah sebagai berikut.

1. Pengguna mengirim permintaan ke URL berawalan `tenant/{tenant:slug}`.
2. *Middleware* autentikasi (`auth`) menentukan identitas pengguna.
3. `SetTenantContext` mengambil tenant dari parameter rute, lalu menolak permintaan bila tenant tidak ditemukan (404), tidak aktif (403), atau pengguna bukan anggotanya (403).
4. `TenantContext` diisi dengan tenant tersebut.
5. Lapisan aplikasi berikutnya — kueri model, *policy*, dan layanan — menggunakan konteks itu.
6. Blok `finally` membersihkan konteks setelah respons selesai, baik permintaan berhasil maupun gagal.

Pemeriksaan keanggotaan pada langkah ketiga penting. Tanpa pemeriksaan itu, pengguna yang sudah masuk cukup mengganti `slug` pada URL untuk memperoleh konteks tenant lain, dan seluruh lapisan berikutnya akan bekerja dengan konteks yang salah.

Middleware ini didaftarkan dengan alias `tenant` dan dipasang pada grup rute tenant di berkas `bootstrap/app.php`.

Kode 4.4 — bootstrap/app.php (potongan grup rute tenant)

```php
Route::middleware(['web', 'auth', 'verified', 'tenant'])
    ->prefix('tenant/{tenant:slug}')
    ->name('tenant.')
    ->scopeBindings()
    ->group(base_path('routes/tenant.php'));

// ...

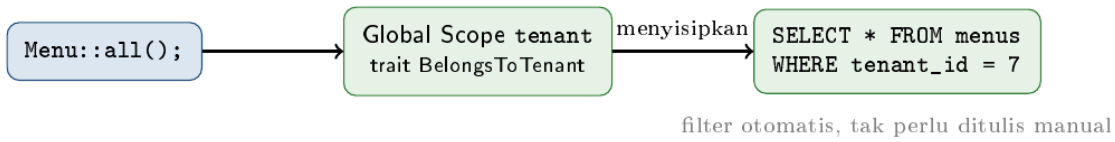
->withMiddleware(function (Middleware $middleware): void {
    $middleware->alias([
        'role' => EnsureUserHasRole::class,
        'tenant' => SetTenantContext::class,
    ]);
})
```

Catatan. Jalur non-HTTP — *queue*, *command*, *scheduler*, atau proses impor data — tidak melewati *middleware* HTTP. Pada jalur tersebut, aplikasi harus menetapkan dan membersihkan `TenantContext` secara eksplisit. Contohnya dibahas pada Subbab 4.6.3.

4.4 Kueri Aman dengan Global Scope

Menentukan tenant aktif belum cukup. Aplikasi juga harus memastikan seluruh model milik tenant menyaring datanya secara otomatis berdasarkan tenant tersebut.

Dalam Laravel, *global scope* adalah aturan yang ditambahkan Eloquent ke setiap kueri sebuah model. Untuk model milik tenant, scope menyisipkan syarat `WHERE tenant_id = ?` tanpa perlu ditulis manual.



Gambar 4.6. Global scope menyisipkan syarat tenant_id ke dalam kueri model.

Sumber: diadaptasi penulis, ilustratif.

4.4.1 Trait BelongsToTenant

Pada proyek pendamping, scope dan pengisian otomatis `tenant_id` dikemas dalam satu *trait* di berkas `app/Models/Concerns/BelongsToTenant.php`.

Kode 4.5 — app/Models/Concerns/BelongsToTenant.php

```php
<?php

namespace App\Models\Concerns;

use App\Support\Tenancy\TenantContext;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;
```

```php
trait BelongsToTenant
{
    public static function bootBelongsToTenant(): void
    {
        static::addGlobalScope('tenant', function (Builder $builder): void {
            $context = app(TenantContext::class);
            if ($context->has()) {
                $builder->where(
                    $builder->qualifyColumn('tenant_id'),
                    $context->id()
                );
            }
        });

        static::creating(function (Model $model): void {
            $context = app(TenantContext::class);
            if ($context->has()
                && empty($model->getAttribute('tenant_id'))) {
                $model->setAttribute('tenant_id', $context->id());
            }
        });
    }
}
```

Cara kerjanya:

- ketika sebuah model seperti `Menu` memakai *trait* `BelongsToTenant`, Laravel mendaftarkan *global scope* bernama `tenant` pada model itu;
- saat aplikasi memanggil `Menu::query()`, scope menambahkan filter `tenant_id` sesuai tenant aktif;
- saat aplikasi membuat menu baru, kolom `tenant_id` diisi dari `TenantContext` apabila belum diberikan;
- scope memberi nama kolom secara lengkap melalui `qualifyColumn()`, sehingga filter tetap tidak ambigu ketika kueri melibatkan `join`.

4.4.2 Fail-Closed: Di Mana Ditegakkan?

Perhatikan bahwa scope pada Kode 4.5 hanya menambahkan filter ketika konteks tersedia (`if ($context->has())`). Secara sekilas, ini tampak melanggar prinsip *fail-closed* yang dinyatakan pada Subbab 4.3. Karena itu, penting memahami di mana prinsip tersebut sebenarnya ditegakkan pada proyek ini.

Jalur operasi	Penegak fail-closed	Perilaku ketika konteks kosong
Rute internal tenant	Middleware `SetTenantContext`	Permintaan ditolak sebelum kueri dijalankan (404/403)
Penulisan data (create)	Kolom `tenant_id` NOT NULL pada basis data	Operasi gagal di tingkat basis data
Keterhubungan antarbaris	Composite foreign key `(tenant_id, x_id)`	Baris lintas tenant ditolak basis data
Otorisasi aksi	`MenuPolicy` dan `TenantOrderPolicy`	`idOrNull()` bernilai null sehingga pembandingan gagal dan aksi ditolak
Pembacaan lintas tenant	Filter eksplisit pada layanan katalog	Kueri wajib menyebut canteen dan

yang sah	publik	status secara eksplisit

Tabel 4.4. Lapisan yang menegakkan prinsip fail-closed pada proyek pendamping.

Dengan kata lain, proyek pendamping menegakkan *fail-closed* melalui pertahanan berlapis — *middleware*, batasan basis data, dan *policy* — bukan melalui pengecualian di dalam *global scope*. Pilihan ini memungkinkan satu model yang sama dipakai pada jalur publik yang memang harus lintas tenant, misalnya katalog kantin yang dilihat pelanggan anonim.

Catatan. Ini adalah **keputusan desain**, bukan satu-satunya cara yang benar. Rancangan alternatif membuat scope memanggil `id()` sehingga kueri tanpa konteks langsung melemparkan pengecualian. Rancangan itu lebih ketat, tetapi menuntut setiap jalur lintas tenant yang sah menonaktifkan scope secara eksplisit. Bandingkan kedua pendekatan pada Latihan nomor 11.

Peringatan. Apa pun rancangan yang Anda pilih, jangan pernah menyimpulkan bahwa "aplikasi aman karena ada *global scope*". Scope hanya menutup satu jalur. Tabel 4.4 harus dapat Anda isi untuk aplikasi Anda sendiri.

4.4.3 Contoh Kueri dan Batasan Global Scope

Dengan *trait* terpasang, pengembang tidak menulis `where('tenant_id', ...)` secara manual.

Kode 4.6 — Contoh 4.3: kueri menu tenant aktif

```
// Kode aplikasi
$menus = Menu::query()->orderBy('name')->get();

// SQL yang secara konseptual dihasilkan
// SELECT * FROM menus
// WHERE menus.tenant_id = :tenant_aktif
// ORDER BY name;
```

Namun, *global scope* bukan jaminan tunggal keamanan. Beberapa kondisi berikut perlu Anda perhatikan.

- Model yang tidak memakai *trait* `BelongsToTenant` tidak akan tersaring otomatis.
- Kueri mentah atau akses basis data yang tidak melalui model Eloquent melewati scope sepenuhnya.
- `withoutGlobalScope('tenant')` menonaktifkan scope secara sengaja.
- Jalur *queue*, *command*, dan *job* harus menetapkan konteks tenant sendiri.
- Laporan lintas tenant yang sah tetap harus dibatasi dengan aturan akses dan audit yang jelas.

Peringatan. Jangan menggunakan `withoutGlobalScope()` sebagai jalan pintas untuk mengatasi kueri yang tidak mengembalikan data. Jika *bypass* memang diperlukan, dokumentasikan alasannya, batasi lokasi penggunaannya pada satu layanan tertentu, tambahkan filter eksplisit yang menggantikan peran scope, dan lindungi aksesnya dengan otorisasi yang sesuai.

4.5 Otorisasi dan Akses Objek

Penyaringan kueri dan otorisasi memiliki tujuan yang berbeda dan tidak saling menggantikan.

- **Penyaringan kueri** menentukan data apa yang dapat **ditemukan** oleh aplikasi.

- **Otorisasi** menentukan tindakan apa yang **boleh dilakukan** pengguna terhadap data yang telah ditemukan.

Sebagai contoh, pengguna tenant 1 hanya dapat melihat menu tenant 1. Namun, tidak semua pengguna tenant 1 otomatis boleh menghapus menu. Hak tersebut tetap perlu diatur melalui *policy*.

4.5.1 Policy Berbasis Kepemilikan Tenant

Policy adalah kelas yang memusatkan aturan otorisasi untuk sebuah model. Pada proyek pendamping, berkasnya adalah `app/Policies/MenuPolicy.php`.

Kode 4.7 — app/Policies/MenuPolicy.php

```php
<?php

namespace App\Policies;

use App\Models\Menu;
use App\Models\User;
use App\Support\Tenancy\TenantContext;

class MenuPolicy
{
    private function ownedByActiveTenant(Menu $menu): bool
    {
        return $menu->tenant_id
            === app(TenantContext::class)->idOrNull();
    }

    public function view(User $user, Menu $menu): bool
    {
        return $this->ownedByActiveTenant($menu);
    }

    public function update(User $user, Menu $menu): bool
    {
        return $this->ownedByActiveTenant($menu);
    }

    public function delete(User $user, Menu $menu): bool
    {
        return $this->ownedByActiveTenant($menu);
    }
}
```

Policy ini membandingkan `tenant_id` menu dengan tenant aktif pada konteks. Penggunaan `idOrNull()` — bukan `id()` — membuat pembandingan tetap menghasilkan `false` ketika konteks kosong, sehingga aksi ditolak alih-alih memicu pengecualian yang sulit ditafsirkan pengguna.

Perhatikan bahwa pembanding adalah **tenant aktif pada konteks**, bukan `tenant_id` milik pengguna. Ini penting karena satu pengguna dapat menjadi anggota lebih dari satu tenant; yang menentukan adalah tenant yang sedang dibuka, bukan seluruh tenant yang pernah ia ikuti.

Dalam aplikasi nyata, Anda biasanya menambahkan aturan berbasis peran — misalnya staf tenant boleh melihat menu tetapi hanya manajer tenant yang boleh menghapusnya. Penambahan peran tidak menggantikan pemeriksaan tenant; keduanya berjalan bersama.

4.5.2 Route Model Binding dan Scoped Binding

Route model binding mengubah parameter URL menjadi objek model. Parameter `{menu}` pada rute diubah Laravel menjadi objek `Menu`.

Pada rute bertingkat seperti `tenant/{tenant:slug}/menus/{menu}`, Laravel dapat mencari `{menu}` melalui relasi model induknya. Perilaku ini diaktifkan dengan `scopeBindings()`, yang pada proyek pendamping dipasang pada grup rute tenant (lihat Kode 4.4). Berkas rutenya adalah `routes/tenant.php`.

Kode 4.8 — routes/tenant.php

```php
<?php

use App\Http\Controllers\Tenant\TenantMenuController;
use App\Models\Tenant;
use Illuminate\Support\Facades\Route;

Route::get('/dashboard', fn (Tenant $tenant) => view('tenant.dashboard', [
    'tenant' => $tenant,
]))->name('dashboard');

Route::get('/menus', [TenantMenuController::class, 'index'])
    ->name('menus.index');
Route::get('/menus/{menu}', [TenantMenuController::class, 'show'])
    ->name('menus.show');
Route::patch('/menus/{menu}', [TenantMenuController::class, 'update'])
    ->name('menus.update');
```

Jika pengguna tenant A membuka `tenant/tenant-a/menus/3` sedangkan menu 3 milik tenant B, dua mekanisme bekerja bersamaan: *scoped binding* membatasi pencarian melalui relasi tenant induk, dan *global scope* membatasi kueri berdasarkan tenant aktif. Keduanya menghasilkan model yang tidak ditemukan, dan Laravel mengembalikan respons 404.

> **Catatan.** *Scoped binding* bukan pengganti *global scope* dan *policy*. Ia membatasi pencarian model melalui relasi rute; *global scope* membatasi kueri model secara lebih luas; *policy* menilai izin tindakan. Ketiganya adalah lapisan yang berbeda pada Gambar 4.4.

[PERLU VERIFIKASI: nama relasi pada model `Tenant` yang dipakai scoped binding untuk parameter `{menu}`, serta perilakunya pada versi Laravel yang digunakan proyek]

4.6 Menguji Isolasi Tenant

Kontrol keamanan yang tidak diuji dapat rusak tanpa disadari ketika kode berubah. Karena itu, aplikasi multi-tenant perlu memiliki pengujian yang secara sengaja mencoba mengakses data tenant lain.

4.6.1 Skenario Uji Minimal

Skenario	Aksi uji	Hasil yang diharapkan
Isolasi daftar	Menghitung `Menu::query()->count()` sebagai tenant A	Hanya menu tenant A yang terhitung
Binding lintas tenant	Membuka menu tenant B melalui rute tenant A	Respons 404
Aksi lintas tenant	Memanggil policy terhadap menu	Policy menolak tindakan

	tenant lain	
Pembuatan data	Membuat data tanpa `tenant_id` eksplisit	`tenant_id` terisi sesuai konteks aktif
Kebersihan konteks	Menyimulasikan siklus request berikutnya	Konteks kosong kembali, tidak terbawa
Konteks pada job	Menjalankan job dua tenant berturut-turut	Hasil tiap job sesuai tenantnya sendiri
Jalur publik	Mengambil katalog satu kantin	Tenant nonaktif dan kantin lain tidak muncul

Tabel 4.5. Skenario pengujian isolasi tenant pada proyek pendamping.

Dua baris terakhir sering terlewat pada pengujian pemula. Baris “kebersihan konteks” membuktikan bahwa `scoped()` benar-benar bekerja, sedangkan baris “jalur publik” membuktikan bahwa jalur yang memang harus lintas tenant tetap memiliki batasannya sendiri.

4.6.2 Contoh Pengujian Isolasi

Proyek pendamping menggunakan PHPUnit dan menjalankan pengujian pada basis data MariaDB nyata. Berkasnya adalah `tests/Feature/TenantIsolationTest.php`.

Kode 4.9 — tests/Feature/TenantIsolationTest.php (tiga metode terpilih)

```php
public function test_global_scope_returns_only_active_tenant_rows(): void
{
    [$a] = $this->tenantWithMenus(2);
    $this->tenantWithMenus(3); // tenant B

    app(TenantContext::class)->set($a);

    $this->assertSame(2, Menu::query()->count());
    $this->assertTrue(
        Menu::query()->get()->every(fn (Menu $m) => $m->tenant_id === $a->id)
    );
}

public function test_scoped_binding_blocks_cross_tenant_menu(): void
{
    [$a] = $this->tenantWithMenus(1);
    [, $menusB] = $this->tenantWithMenus(1);
    $menuB = $menusB->first();

    $this->actingAs($this->memberOf($a));

    $this->getJson(route('tenant.menus.show', [
        'tenant' => $a->slug,
        'menu' => $menuB->id,
    ]))->assertNotFound();
}

public function test_context_is_reset_between_requests(): void
{
    [$a] = $this->tenantWithMenus(1);
    app(TenantContext::class)->set($a);
    $this->assertTrue(app(TenantContext::class)->has());

    $this->app->forgetScopedInstances();

    $this->assertFalse(
```

```
        app(TenantContext::class)->has(),
        'TenantContext bocor antar request/job'
    );
}
```

Setiap metode uji mengikuti tiga tahap yang sama.

1. *Arrange*: menyiapkan dua tenant beserta menunya melalui *helper* `tenantWithMenus()`.
2. *Act*: menetapkan tenant aktif, atau mengirim permintaan HTTP sebagai anggota tenant tertentu.
3. *Assert*: memastikan data tenant lain tidak terbaca, tidak dapat dibuka, dan konteks tidak terbawa.

Jalankan seluruh berkas uji ini di terminal dengan perintah berikut.

Kode 4.10 — Menjalankan pengujian isolasi (di terminal)

```
php artisan test --filter=TenantIsolationTest
```

4.6.3 Konteks Tenant pada Jalur Non-HTTP

Sebuah *job* tidak melewati *middleware* HTTP, sehingga harus menetapkan dan membersihkan konteksnya sendiri. Berkasnya adalah `app/Jobs/CountTenantMenus.php`.

Kode 4.11 — app/Jobs/CountTenantMenus.php

```
public function __construct(public int $tenantId) {}

public function handle(TenantContext $context): void
{
    $tenant = Tenant::query()->findOrFail($this->tenantId);
    $context->set($tenant);

    try {
        // Query memakai global scope tenant aktif.
        $this->result = Menu::query()->count();
    } finally {
        $context->clear();
    }
}
```

Perhatikan dua keputusan penting pada kode ini. Pertama, *job* membawa `tenantId` berupa bilangan, bukan objek `Tenant` atau konteks yang sudah terbentuk; dengan begitu konteks selalu dibangun ulang di sisi *worker*. Kedua, blok `finally` membersihkan konteks meskipun terjadi kegagalan, sehingga *job* berikutnya pada *worker* yang sama tidak mewarisi tenant sebelumnya.

4.6.4 Kriteria Evaluasi

Kriteria	Pertanyaan evaluasi	Indikator minimum
Cakupan model	Apakah semua model milik tenant memakai mekanisme isolasi?	Setiap model relevan memakai trait atau kontrol setara
Fail-closed	Apakah konteks kosong menyebabkan operasi ditolak?	Tabel 4.4 dapat diisi untuk setiap jalur operasi
Otorisasi aksi	Apakah operasi tulis memanggil policy atau mekanisme setara?	Aksi penting memiliki pengujian otorisasi

Audit bypass	Apakah penggunaan `withoutGlobalScope()` terdokumentasi?	Setiap bypass memiliki alasan, pembatasan, dan pengujian
Cakupan pengujian	Apakah semua skenario pada Tabel 4.5 diuji?	Skenario lulus pada integrasi berkelanjutan
Jalur non-HTTP	Apakah job, queue, dan command mengatur konteks tenant?	Konteks ditetapkan dan dibersihkan pada setiap unit kerja

Tabel 4.6. Kriteria evaluasi isolasi tenant.

4.7 Studi Kasus Hipotetis: Laporan Lintas Tenant

Bagian ini menyatukan konsep yang telah dipelajari melalui sebuah contoh **hipotetis**.

Tim pengembang aplikasi Kantin memilih *shared database–shared schema* karena aplikasi ditujukan untuk banyak tenant kecil dengan struktur menu dan pesanan yang seragam. Setiap tabel milik tenant memiliki kolom `tenant_id`.

Pada awal pengembangan, halaman laporan menggunakan `withoutGlobalScope('tenant')` untuk menghitung data lintas tenant. Kode tersebut kemudian dipakai ulang pada halaman yang dapat diakses operator tenant. Akibatnya, halaman laporan berisiko menampilkan data tenant lain karena tidak ada filter pengganti dan otorisasi yang tepat.

Tim memperbaiki rancangan dengan langkah berikut.

1. Menetapkan `TenantContext` melalui *middleware* `SetTenantContext` setelah autentikasi, disertai pemeriksaan keanggotaan.
2. Mendaftarkan `TenantContext` sebagai *scoped binding* agar tidak terbawa antar-permintaan.
3. Menerapkan *trait* `BelongsToTenant` pada seluruh model milik tenant.
4. Menambahkan *policy* pada aksi pembacaan, pembaruan, dan penghapusan.
5. Memindahkan seluruh *bypass* scope ke satu layanan khusus yang memiliki filter eksplisit dan otorisasi tersendiri.
6. Menambahkan pengujian untuk daftar data, akses lintas tenant melalui URL, *policy*, kebersihan konteks, dan konteks pada *job*.

Hasil yang **diharapkan** dari perbaikan tersebut adalah pengguna tenant hanya dapat melihat dan mengubah data tenantnya sendiri pada jalur aplikasi yang diuji. Namun, tim tetap perlu mengaudit kueri mentah, proses ekspor data, jalur *queue*, dan setiap *endpoint* baru.

> **Peringatan.** Skenario di atas adalah studi kasus hipotetis yang disusun untuk keperluan pembelajaran, bukan laporan hasil implementasi empiris. Angka, urutan kejadian, dan hasil perbaikan tidak berasal dari pengukuran nyata.

Rangkuman Bab

- *Multi-tenancy* memungkinkan satu aplikasi melayani banyak tenant, tetapi menuntut isolasi data yang dirancang secara sadar.
- Tiga strategi utama penyimpanan adalah *shared database–shared schema*, *shared database–separate schema*, dan *separate database*. Tidak ada strategi yang otomatis paling aman atau paling murah pada semua kondisi.
- Pada *shared schema*, kolom `tenant_id` dan *tenant context* menjadi fondasi isolasi data.

- `TenantContext` didaftarkan sebagai *scoped binding* agar tidak terbawa antar-permintaan atau antar-*job*.
- *Middleware* menentukan tenant aktif sekaligus memverifikasi keanggotaan pengguna, lalu membersihkan konteks pada blok `finally`.
- *Global scope* menyaring kueri secara otomatis, tetapi tetap harus diaudit karena dapat dilewati oleh model tertentu, kueri mentah, atau *bypass* yang disengaja.
- Prinsip *fail-closed* ditegakkan berlapis: *middleware*, batasan `NOT NULL` dan *composite foreign key* pada basis data, serta *policy* — bukan oleh satu mekanisme tunggal.
- *Policy* membatasi tindakan pengguna terhadap sumber daya, sehingga penyaringan data dan otorisasi bekerja sebagai pertahanan berlapis.
- Pengujian lintas tenant diperlukan untuk membuktikan bahwa isolasi tetap bekerja ketika aplikasi berkembang.

Bab berikutnya membahas identitas dan hak akses pengguna: registrasi, autentikasi, otorisasi berbasis peran, operasi data tervalidasi, serta administrasi tenant dan skema komisi — dengan tetap mempertahankan prinsip isolasi dan pengujian yang telah dibangun pada bab ini.

Istilah Penting

Istilah	Makna ringkas
Multi-tenancy	Satu instans aplikasi melayani banyak penyewa
Tenant	Satu penyewa; pada buku ini, satu tenant kantin
Shared schema	Seluruh tenant berbagi tabel yang sama
Tenant context	Objek penyimpan identitas tenant aktif satu siklus kerja
Scoped binding	Pendaftaran container: satu instans per request atau job
Tenant resolution	Proses menentukan tenant yang sedang aktif
Global scope	Aturan yang Eloquent tambahkan ke setiap kueri model
Fail-closed	Menolak operasi ketika identitas tenant tidak dapat dipastikan
Policy	Kelas pemusat aturan otorisasi untuk sebuah model
Scoped binding rute	Pencarian model anak melalui relasi model induk pada rute
Composite foreign key	Kunci asing gabungan yang mengikat baris ke tenant yang sama
Defense in depth	Pertahanan berlapis; satu lapis gagal, lapis lain menahan

Tabel 4.7. Istilah penting pada Bab 4.

Pertanyaan dan Latihan

A. Pemahaman

1. Jelaskan perbedaan *multi-tenancy* dan *single-tenancy* menggunakan contoh aplikasi kantin.
2. Mengapa kolom `tenant_id` menjadi penting pada strategi *shared database–shared schema*?
3. Jelaskan perbedaan fungsi *tenant context*, *global scope*, dan *policy*.
4. Apa yang dimaksud dengan prinsip *fail-closed*, dan mengapa prinsip ini penting pada aplikasi multi-tenant?

5. Mengapa `TenantContext` didaftarkan dengan `scoped()`, bukan `singleton()`? Pada kondisi seperti apa perbedaannya terasa?

B. Penerapan

6. Sebuah aplikasi memiliki tabel `orders` yang menyimpan pesanan dari 20 tenant. Jelaskan langkah yang Anda lakukan agar model `Order` selalu dibatasi oleh tenant aktif.
7. Tulis rancangan pengujian untuk memastikan pengguna tenant A tidak dapat memperbarui menu milik tenant B melalui rute HTTP.
8. Berdasarkan Persamaan 4.2, hitung beban operasional ilustratif untuk 120 tenant apabila setiap tenant menggunakan basis data terpisah dan $b = 1$. Bandingkan dengan *shared schema* yang menggunakan satu skema.
9. Berdasarkan Persamaan 4.1, hitung perkiraan jumlah baris milik tenant lain yang terekspos apabila tabel `orders` memuat 4.000 baris dari 25 tenant.
10. Tambahkan aturan berbasis peran pada `MenuPolicy` sehingga hanya anggota berperan `manager` yang boleh menghapus menu, tanpa menghilangkan pemeriksaan tenant yang sudah ada.

C. Analisis dan Evaluasi

11. Bandingkan dua rancangan *global scope*: (a) scope yang hanya memfilter ketika konteks tersedia, seperti pada Kode 4.5; dan (b) scope yang melemparkan pengecualian ketika konteks kosong. Uraikan keunggulan, kelemahan, dan kondisi ketika masing-masing lebih sesuai.
12. Sebuah tim menggunakan `withoutGlobalScope('tenant')` pada fitur ekspor laporan. Analisis risiko yang mungkin muncul dan usulkan tiga kontrol untuk menguranginya.
13. Anda menemukan *endpoint* API yang memakai kueri mentah tanpa filter `tenant_id`. Jelaskan mengapa kondisi ini berbahaya dan tuliskan strategi perbaikannya.
14. Bandingkan pilihan *shared schema* dan *separate database* untuk dua kondisi: (a) aplikasi untuk 5.000 tenant kecil dengan struktur data seragam; (b) aplikasi untuk 10 organisasi besar yang masing-masing membutuhkan struktur data berbeda dan pemisahan operasional lebih ketat. Berikan alasan untuk setiap pilihan.
15. Rancang daftar uji minimum untuk memastikan isolasi tenant tetap aman pada permintaan web, API, *queue*, dan proses ekspor data. Gunakan Tabel 4.6 sebagai kerangka.
16. Middleware `SetTenantContext` memeriksa keanggotaan pengguna sebelum mengisi konteks. Analisis apa yang dapat terjadi apabila pemeriksaan itu dihapus, meskipun *global scope* dan *policy* tetap terpasang.

Daftar Pustaka

Buku

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly Media.

Stauffer, M. (2023). *Laravel: Up & Running* (3rd ed.). O'Reilly Media.

Dokumentasi dan Standar (Daring)

Laravel LLC. (2026). *Eloquent: Global Scopes, Authorization: Policies, Routing: Route Model Binding — Laravel 13.x Documentation*. Diakses dari https://laravel.com/docs/13.x/eloquent

OWASP Foundation. (2021). *OWASP Top 10:2021 — A01 Broken Access Control*. Diakses dari https://owasp.org/Top10/A01_2021-Broken_Access_Control/

[PERLU VERIFIKASI: metadata, URL, tanggal akses, edisi, dan DOI seluruh rujukan daring]

Sumber Proyek (Dokumen Internal dan Repositori)

Suwardiyanto, D. (2026). *Kantin Multi-Tenant — Repositori Kode Pendamping*, tag `pertemuan-04`. https://github.com/devitsuwardiyanto/kantin-multi-tenant

Politeknik Negeri Banyuwangi. *SRS Aplikasi Kantin Multi-Tenant (ISO/IEC/IEEE 29148), versi 2*. Dokumen proyek.

Politeknik Negeri Banyuwangi. *Modul Praktikum Kantin Multi-Tenant Laravel 13 — Modul 4: Isolasi Multi-Tenant*. Dokumen praktikum.

[PERLU VERIFIKASI: penomoran kebutuhan pada SRS (misalnya FR-TEN dan UC-19) serta status dan izin penggunaan dokumen internal dalam buku ajar]

Catatan: seluruh data numerik, diagram, dan studi kasus pada bab ini bersifat ilustratif untuk keperluan pembelajaran. Seluruh potongan kode telah diselaraskan dengan repositori pendamping pada tag pertemuan-04.